

1. Différence entre système distribué et système parallèle

- **Système distribué** : Ensemble de machines indépendantes interconnectées via un réseau. Chaque machine a sa propre mémoire. Exemple : Cloud computing, systèmes bancaires.
 - **Système parallèle** : Plusieurs processeurs (ou cœurs) partagent la mémoire et exécutent des tâches simultanément. Exemple : supercalculateurs.
-

2. Définir l'architecture du système distribué

- **Client/Serveur** : Le client envoie une requête, le serveur répond.
 - **Peer-to-Peer (P2P)** : Tous les nœuds peuvent jouer le rôle de client et de serveur.
 - **Architecture n-tiers** : Séparation logique (présentation, logique métier, données).
 - **Microservices** : Applications divisées en petits services indépendants interconnectés.
-

3. Les défis et les avantages du système distribué

- **Avantages** :
 - Tolérance aux pannes
 - Scalabilité
 - Répartition de charge
 - Partage de ressources
 - **Défis** :
 - Synchronisation
 - Fiabilité
 - Sécurité
 - Cohérence des données
 - Latence réseau
-

4. Différence entre système 1 cœur, multicœurs et multiprocesseurs

- **1 cœur** : Un seul processeur exécute les instructions.
 - **Multicœurs** : Un seul processeur physique contient plusieurs cœurs (unités de traitement).
 - **Multiprocesseurs** : Plusieurs processeurs physiques dans une même machine.
-

5. Définir le modèle de fonctionnement et modèle de programmation

- **Modèle de fonctionnement** : Décrit la manière dont les composants interagissent (message, appel distant, etc.).
 - **Modèle de programmation** : Ensemble d'abstractions et d'outils permettant de développer des applications distribuées (RMI, RPC, Actors, etc.).
-

6. Protocoles TCP vs UDP

- **TCP (Transmission Control Protocol)** :
 - Connexion établie
 - Fiable (accusé de réception, retransmission)
 - Ordre garanti
 - Utilisé pour HTTP, FTP, etc.
 - **UDP (User Datagram Protocol)** :
 - Sans connexion
 - Pas de garantie de livraison ou d'ordre
 - Utilisé pour DNS, VoIP, streaming vidéo
-

7. Sérialisation

- **Définition** : Processus de conversion d'un objet en une suite d'octets pour le stockage ou la transmission.
- **Types de représentation** :
 - Binaire (Java Serializable)

- Texte (XML, JSON, YAML)
 - Protocol Buffers (Google)
-

8. Middleware RPC (Remote Procedure Call)

- Permet à un programme d'exécuter une procédure sur un autre ordinateur comme s'il s'agissait d'un appel local.
 - Composants :
 - Interface
 - Stub client
 - Stub serveur (Skeleton)
 - Registre (annuaire)
 - Processus : appel → sérialisation → transmission → exécution → réponse
-

9 et 10. Définition et principe du RMI & Design Pattern Proxy

- **RMI (Remote Method Invocation) :**
 - Permet d'invoquer des méthodes sur des objets distants en Java.
 - Basé sur le modèle RPC.
- **Composants RMI :**
 - Interface distante
 - Implémentation
 - Stub (côté client)
 - Skeleton (côté serveur)
 - Registry (annuaire)
- **Design pattern Proxy :**
 - Le stub agit comme un **proxy local** de l'objet distant.
 - Masque la complexité de l'appel distant.

